

Task Management Platform - Complete Architecture & Interview Guide

1. Tech Stack

FastAPI (REST APIs), PostgreSQL (persistent storage), Redis (caching), SQLAlchemy ORM, JWT Authentication, Docker & Docker Compose, Pydantic validation.

2. High Level Architecture

Client -> FastAPI -> Service Layer -> SQLAlchemy -> PostgreSQL.

Redis stores frequently accessed data. JWT secures endpoints. Docker runs every service in isolated containers.

3. Suggested Folder Structure

```
app/  
api/  
auth.py  
users.py  
tasks.py  
services/  
repositories/  
models/  
schemas/  
core/  
db/  
utils/  
main.py
```

4. Features

- User Registration/Login
- JWT Access Token authentication
- Role Based Access Control (Admin/User)
- Create, Update, Delete Tasks
- Assign tasks to users
- Task status (Todo/In Progress/Done)
- Pagination
- Filtering
- Search
- Input validation
- Global exception handling
- Redis caching
- Dockerized deployment

5. Database Tables

users(id,name,email,password_hash,role)

tasks(id,title,description,status,assigned_to,created_by,created_at,updated_at)

6. API Endpoints

POST /register

POST /login

GET /tasks
POST /tasks
GET /tasks/{id}
PUT /tasks/{id}
DELETE /tasks/{id}
PATCH /tasks/{id}/status

7. Authentication

Passwords hashed using bcrypt. Login returns JWT. Protected endpoints verify JWT before execution. RBAC ensures only authorized users perform admin operations.

8. Redis Usage

Cache task list and frequently accessed task details. Use TTL for cache expiration. Invalidate cache after create/update/delete.

9. SQLAlchemy

Models map Python classes to PostgreSQL tables. Repository layer performs CRUD operations using ORM instead of raw SQL.

10. Docker

docker-compose runs FastAPI, PostgreSQL and Redis together. Environment variables store secrets and database URLs.

11. Request Flow

Client sends request -> JWT validation -> Router -> Service -> Repository -> PostgreSQL -> Response.

If cached data exists, Redis responds before hitting database.

12. Possible Improvements

Background jobs with Celery/Kafka, Notifications, File attachments, Audit logs, Soft delete, Rate limiting, CI/CD pipeline, AWS deployment, Kubernetes deployment.

13. Interview Questions

Why Redis?
Why SQLAlchemy?
Why JWT instead of Sessions?
Why PostgreSQL?
How is RBAC implemented?
How does pagination work?
How do you invalidate cache?
Why Docker Compose?
How would you scale this application?
How would you introduce Kafka?

14. Resume Talking Points

Mention layered architecture, REST API design, JWT authentication, RBAC, SQLAlchemy ORM, Redis caching, Docker, pagination, filtering, search, exception handling, modular code structure, and clean architecture.